

¿DÓNDE ESTÁ? × NEONSTRIDE

Documento técnico del videojuego web

Integrantes: Diego Heredia y Jenmarie Polanco

Asignatura: Desarrollo e implementación de soluciones web y multimedia

Docente: Carlos Escalante

Institución: Politécnico San Valero

1 Descripción del proyecto

El proyecto integra un juego de percepción visual con una carrera contra el rival virtual PRISM. Cada ronda contiene una cuadrícula de elementos iguales y exactamente un intruso. El jugador debe identificarlo antes de perder la carrera. Las preguntas educativas ofrecen oportunidades limitadas de recuperación.

La refactorización conserva el estilo oscuro y neón, los avatares, las pistas, los temas y los efectos. Su principal cambio técnico es separar las reglas del juego de la interfaz y registrar jugadores y partidas en una base de datos real. Práctica permite entrenar sin presión y queda excluida de la clasificación competitiva.

2 Objetivos

Desarrollar una aplicación funcional, modular y explicable por dos integrantes. Los objetivos específicos son desacoplar el motor del DOM, centralizar la configuración, validar las entradas, persistir resultados mediante REST y comprobar las reglas con pruebas automatizadas.

La organización limita los archivos de código a 300 líneas, las funciones a 40 y la anidación a tres niveles. Cada módulo debe tener una responsabilidad identificable para facilitar mantenimiento, estudio y defensa individual.

3 Tecnologías utilizadas

Tecnología	Responsabilidad
HTML5 y CSS3	Estructura, modales, temas y diseño responsivo.
JavaScript ES6	Motor, entrada, renderizado y servicios HTTP.
Node.js y Express	API REST, rutas y middleware.
SQLite	Perfiles, partidas, restricciones e índices.
Web Audio	Sonidos generados sin archivos externos.
ESLint, Prettier y Acorn	Estilo y auditoría de la estructura.
Node Test, JSDOM y Playwright	Pruebas de dominio, API, interfaz y navegador.

Arquitectura y organización

4 Separación de responsabilidades



GameEngine recibe acciones y tiempo transcurrido, actualiza sus entidades y expone un snapshot. No utiliza document ni modifica elementos HTML. InputManager traduce eventos a acciones. Las vistas presentan los datos y Application coordina pantallas y ciclo de vida; main.js solo inicia la aplicación.

El servidor sigue route → controller → service → repository → database. Las rutas enlazan URL y controller. Los controllers traducen HTTP. Los services validan y autorizan sin req/res. Los repositories ejecutan SQL parametrizado. server.js inicia el servicio; app.js compone las dependencias.

5 Diagrama de carpetas

```
neonstride/  
  frontend/  
    index.html  
    src/  
      css/      base, layout, components, game, responsive  
      js/  
        main.js  
        core/    entities/    states/  
        input/   modules/     services/  
        utils/   data/        ui/  
        assets/  images, sounds, fonts  
  backend/  
    server.js  
    src/  
      config/    routes/      controllers/  
      services/  repositories/ models/  
      middlewares/ utils/  
  tests/    scripts/    docs/
```

Los módulos board, scoring, difficulty y quiz encapsulan reglas distintas; data conserva símbolos y preguntas. Los estados separan menú, selección, juego, pausa, resultados, récords, logros, configuración y ayuda. Los servicios del frontend distinguen HTTP del almacenamiento de preferencias.

Datos y comunicación REST

6 Diagrama de base de datos



7 Entidades y persistencia

Cada jugador puede registrar muchas partidas. El perfil almacena un hash de su token; la API no publica ese secreto. `submission_id` evita duplicar una partida cuando se reintenta un envío. Los récords y catorce logros se calculan desde el historial competitivo, sin duplicar ese estado en otras tablas.

En memoria, Player conserva puntuación, vidas, pistas, racha, nivel y progreso; Prism calcula el avance del rival; Quiz reúne pregunta y reloj. La base aplica claves foráneas y restricciones. El esquema se crea de forma idempotente y el archivo SQLite sobrevive al reinicio del servidor.

8 API REST y seguridad

Método y ruta	Operación
GET / POST /api/players	Consultar y registrar perfiles.
PUT /api/players/:id	Editar con token del propietario.
GET / POST /api/games	Consultar y guardar partidas.
GET /api/players/:id/games	Consultar partidas del piloto.
GET /api/players/:id/statistics	Obtener récords y logros.
GET /api/leaderboard	TOP 5 filtrable por modo y dificultad.

Las respuestas incluyen `success`, `data` y `message`. Se usan códigos 200, 201, 400, 401, 404, 409, 413 y 500. La validación comprueba tipos, límites y coherencia; el servidor recalcula la precisión. CORS permite orígenes explícitos y el middleware global oculta detalles internos de errores.

Flujo y comprobación del juego

9 Flujo principal y progresión

Etapa	Comportamiento
Inicio	Registrar perfil, elegir modo y dificultad y generar un intruso.
Ronda	Avanzan el reloj y PRISM; el jugador selecciona una celda.
Acierto	Sumar puntos, racha y progreso; frenar a PRISM; subir de nivel cada tres aciertos.
Error o timeout	Penalizar puntos, romper racha, mostrar respuesta y perder una vida.
Recuperación	Si PRISM llega o faltan vidas: pregunta de cuatro opciones, 12 segundos y hasta dos oportunidades.
Final	Victoria al 100 %; derrota al agotar recuperación; guardar resultado y actualizar estadísticas.

En cada nivel PRISM aumenta su velocidad con una curva cuadrática y los aciertos lo frenan menos. La dificultad seleccionada amplifica la presión. El tiempo baja hasta un mínimo de 4 segundos y el tablero crece hasta 8×8 . A igual progreso, su velocidad en el nivel 10 es aproximadamente 5,46 veces la del nivel 1.

Una respuesta educativa correcta garantiza al menos una vida y devuelve a PRISM al 35 %, sin reiniciar el nivel. Práctica elimina cronómetro, presión y pérdida de vidas, mantiene pistas ilimitadas y no modifica récords ni logros competitivos. H ofrece una pista, P pausa y M controla sonido.

10 Verificación y conclusión

La versión comprobada supera 31 pruebas de motor, API e integración DOM. Se verifican los límites de código con Acorn y ESLint y el formato con Prettier. Las pruebas incluyen progresión, victoria, recuperación, pausa, persistencia, autorización y reintento sin duplicados.

La revisión visual en navegador sigue pendiente por las restricciones de ejecución del entorno. El DOM simulado comprueba comportamiento, pero no certifica el diseño ni el sonido en dispositivos reales. El proyecto incorpora un script de prueba visual y un checklist para finalizar esa revisión.

La arquitectura permite estudiar cada responsabilidad sin incorporar un framework de interfaz ni un ORM. Para desplegar es necesario conservar SQLite en un volumen persistente. La validación de estadísticas no reemplaza un sistema antitrampas basado en reproducción de partidas en servidor.

La entrega se completa con las aportaciones y commits de ambos integrantes, las capturas, la publicación, el video de 5 a 8 minutos y la defensa individual. El repositorio local está inicializado; no se atribuyen commits ficticios.